From Rules to Runtime

Jonas Walcher
TUM School of Computation, Information and Technology

Abstract

LLM agents no longer just generate text; they take actions, call tools, and modify real systems. We propose a runtime guardrail that maps natural-language policies to the specific tools they govern, then intercepts each tool call with only the relevant policy context. Across airline, telecom, and retail tasks from τ^2-bench, this reduces policy violations by up to 22.2 percentage points and improves reward for most models. The results show that focused runtime enforcement can make tool-using agents safer, but also exposes the hard tradeoff: more compliance comes with added latency, false positives, and messy guard-agent negotiation.

Keywords: LLM agents, runtime guardrails, policy compliance, tool use, agent safety, runtime enforcement

Introduction

Motivation. LLM agents are increasingly able to act through external tools, APIs, and databases. As a result, failures are no longer limited to wrong text outputs. A tool-using agent can update a booking, modify customer data, issue a refund, or execute another state-changing action. This makes preventative safeguards especially important in policy-constrained domains such as airline, telecom, and retail workflows.

Problem. The central problem in this work is how to design and evaluate a guardrail architecture that keeps tool-using LLM agents policy-compliant. Such an architecture must satisfy three requirements:

1. It must be flexible across domains with different policies and tools.
2. It must scale to long policy documents and many tool calls.
3. It must prevent unauthorized actions without blocking valid task completion.

This is difficult because business policies are usually long, domain-specific, and written in natural language. Also, not every policy rule is relevant to every tool call. Giving the full policy document to every guard is therefore inefficient and can make the guard less reliable. In our preliminary experiment, guard recall decreases when irrelevant policy context is added, especially for smaller models. This suggests that policy context selection is not only an efficiency issue, but also a safety issue.

Related work and gap. Prior work has approached agent safety from several directions. First, formal frameworks define explicit rules or constraints for agent behavior, for example through DSL-based runtime rules, verifiable policy reasoning, or temporal constraints for tool use (Chen et al., 2025; Doshi et al., 2026; Wang et al., 2025). Second, adaptive and training-based guardrails generate safety checks over time, train on synthetic risk scenarios, post-train models for safer tool-use decisions, or personalize protection to users and interaction history (Agarwal et al., 2026; Luo et al., 2025; Wu et al., 2025; Zhou et al., 2025). Third, runtime enforcement systems monitor tool calls before execution. ToolGuard, the closest prior work, maps natural-language company policies to tools and compiles them into deterministic guard code (Zwerdling et al., 2025). Other systems use guard agents, step-level feedback, task constraints, stateful runtime protection, or provenance-aware monitors (Chinaei, 2026; H. Liu et al., 2026; Mou et al., 2026; Xiang et al., 2024; Zhao et al., 2026).

These works show that runtime enforcement is a promising direction. However, systems such as ToolGuard mainly compile mapped policies into deterministic guard code, which can be limiting when compliance depends on context that is hard to encode explicitly. We instead use automatically mapped, tool-specific policy passages as runtime context for an independent LLM guard. This is more flexible and allows feedback to the agent, but is also probabilistic and harder to verify. This tradeoff is the gap we study in this paper.

Approach. We propose and evaluate a complete guard architecture for policy-compliant LLM agents. The architecture has three parts:

1. **Policy-to-tool mapper.** The mapper decomposes a policy document into atomic constraint statements and binds each statement to the tools it governs. We compare an LLM-based mapper optimized for accuracy with a retrieval-based mapper using BM25, a bi-encoder, a cross-encoder, and an LLM judge for scalability.
2. **Intercepting LLM guard.** Before each tool call, the guard receives the conversation history, the proposed tool call, and only the policy passages mapped to that tool. It returns a structured verdict: *approve*, *reject*, or *escalate*. If it rejects a call, it also gives feedback to the agent, creating a live compliance dialogue rather than a simple binary gate.
3. **Multi-domain evaluation.** We evaluate the system on

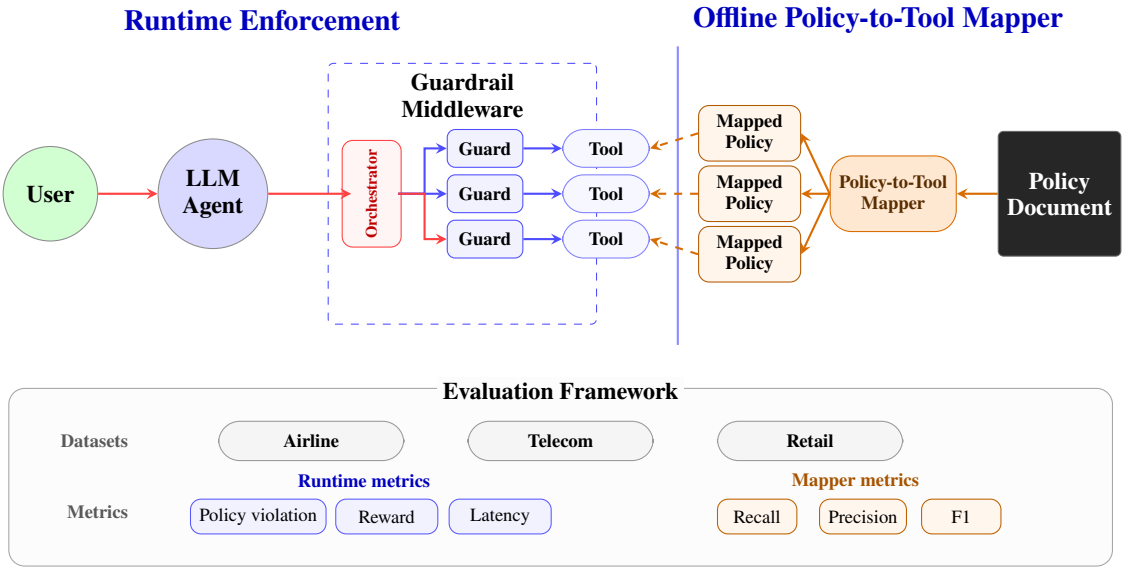

Figure 1: Architecture and evaluation overview. Offline mapping assigns policy passages to tools; runtime middleware checks tool calls with mapped context. Evaluation covers three domains using runtime metrics and mapper precision/recall/F1.

τ^2-bench across airline, telecom, and retail (Barres et al., 2025). We use these benchmarks because they provide realistic tool-use tasks and established benchmark scores, and we extend them with additional policy-compliance tasks and evaluation criteria.

Results. Runtime guards reduce policy violations across all domains, but the gains come with added latency, low mapper precision, and many false positives. The results support targeted policy context while showing that calibration remains the main deployment challenge.

Contributions. This work makes four contributions:

1. We identify policy-context length as a concrete problem for LLM-based guards.
2. We propose and evaluate a policy-to-tool mapping pipeline that assigns relevant policy passages to each tool.
3. We introduce an intercepting LLM guard that uses mapped policy passages as runtime context and gives structured feedback to the agent. This middleware architecture can be integrated freely in other projects together with a full stack web-app [1]
4. We extend τ-bench with additional policy-compliance tasks and evaluate the full architecture across airline, telecom, and retail using policy violation rate, task reward, and latency.

Policy-to-Tool Mapper

The offline policy-to-tool mapper links natural-language policy passages to the tool-level guards that need them. This avoids the naive alternative of sending the full policy document to every guard, which is expensive, does not scale to long policies, and can make relevant rules harder to identify due to irrelevant context and long-context effects such as "lost in the middle" (N. F. Liu et al., 2024).

[1]Code and data are available at https://github.com/byjonny/compliant_agents.

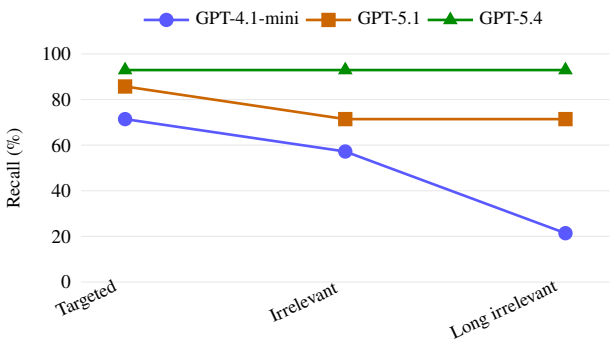

Figure 2: Recall of policy violation detection under increasing policy-context length. All results use 14 test cases over two runs.

To test this issue, we ran a preliminary experiment in the airline domain. Each test case contained a simulated user-agent conversation that ended with a proposed tool action violating an airline policy. The LLM's task was only to detect whether the proposed tool action was policy-compliant. We compared three context settings: (1) targeted policy text, (2) larger policy text with irrelevant passages, and (3) long policy text with even more irrelevant passages.

The results in Figure 2 support the hypothesis that irrelevant policy context can reduce guard reliability, especially for smaller models. For GPT-4.1-mini, recall dropped from

71.4% to 57.2% and then to 21.4%. GPT-5.1 also degraded, from 85.7% to 71.4%, while GPT-5.4 stayed stable at 92.9%. We therefore use the policy-to-tool mapper to reduce context noise and provide focused runtime context for the guard.

Mapper Architectures

The mapper takes a policy document and an OpenAPI tool specification as input, using OpenAPI as a portable, language-agnostic interface for REST tools (Lercher et al., 2024; OpenAPI Initiative, 2017).

For each protected write tool, the mapper returns the policy passages that should be given to the runtime guard. We define an atomic policy statement as a self-contained rule that limits, conditions, or forbids tool use. We focus on write tools because they can change system state and unauthorized write actions can be evaluated clearly from execution logs.

Domain	Write tools with relevant passages	Relevant passages
Airline	6	50
Retail	8	35
Telecom	7	17

Table 1: Policy-to-tool mapping scope by domain. We only evaluate write tools with policy passages relevant to runtime enforcement.

We evaluate two mapper architectures: (1) an LLM-based mapper optimized for accuracy, and (2) a retrieval-based mapper optimized for scalability.

LLM-based mapper. The LLM-based mapper has four stages: the Chunker extracts atomic policy statements from policy markdown; the Profiler summarizes each tool's purpose, inputs, side effects, and policy categories; the Mapper assigns policy statements to tools with high/medium confidence labels; and the Sweeper re-checks sparsely mapped tools for missing rules. We report only high-confidence mappings.

The main limitation of this architecture is scalability. The mapper sends the full statement list to the LLM once per tool, resulting in $O(\text{tools} \times \text{statements})$ token cost. This is manageable in our current setting, but grows linearly with policy size and number of tools.

Retrieval-based mapper. The retrieval-based architecture was introduced because the LLM-based mapper becomes expensive as tools and policy statements grow. It keeps the same Chunker, Profiler, and Sweeper, but replaces full LLM matching with retrieval followed by an LLM judge.

The retriever has three parts. The retrieval mapper keeps the same Chunker, Profiler, and Sweeper, but first retrieves candidate policy statements with BM25 and text-embedding-3-large, reranks them with ms-marco-MiniLM-L-6-v2, and sends only the top candidates to an LLM judge.

For each tool profile, BM25 and the bi-encoder each retrieve the top 30 policy statements. The cross-encoder reranks candidates so only the top-ranked ones are passed to the LLM judge. This improves scalability by reducing policy context, but creates a recall risk: if retrieval or reranking removes a relevant rule too early, the LLM judge cannot recover it.

Evaluation

We evaluated both mapper architectures against a manually constructed ground truth. The ground truth was built in three steps: (1) the policy was decomposed into atomic constraint units (Rabinovich et al., 2026), (2) each unit was bound to tools based on whether it directly governs when or how a tool may be called, and (3) all bindings were reviewed in a final manual pass.

We evaluate both mappers against manually constructed ground truth: policies are decomposed into atomic constraint units, each unit is bound to tools it directly governs, and all bindings are manually reviewed (Rabinovich et al., 2026). We report micro-averaged precision and recall because each policy-tool binding is a concrete runtime decision: missing a rule removes relevant guard context, while adding a wrong one introduces irrelevant context. For matching predicted and ground-truth passages, we use literal overlap instead of embedding similarity, since cosine similarity can hide small but important differences in rule coverage.

Results

Figure 4 shows the domain-macro average of micro recall across the three domains for high-confidence mappings. For each model and architecture, we first compute micro recall within each domain and then average the three domain-level scores.

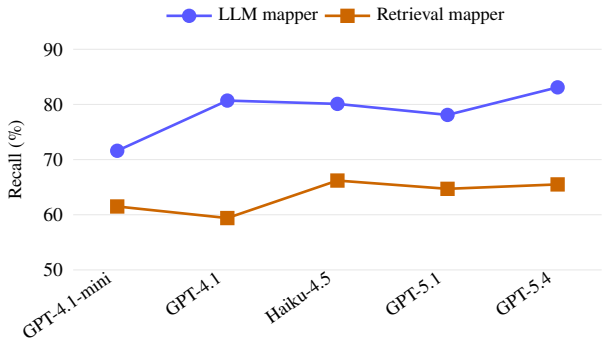

Figure 4: Domain-macro average of micro recall across retail, airline, and telecom for high-confidence mappings. Each domain is weighted equally.

The LLM-based mapper achieves higher recall, ranging from 71.6% to 83.1%, while the retrieval-based mapper ranges from 59.4% to 66.2%. Retrieval loses between 10.0 and 21.3 percentage points depending on the model. This shows the main scalability–recall tradeoff: retrieval reduces the num-

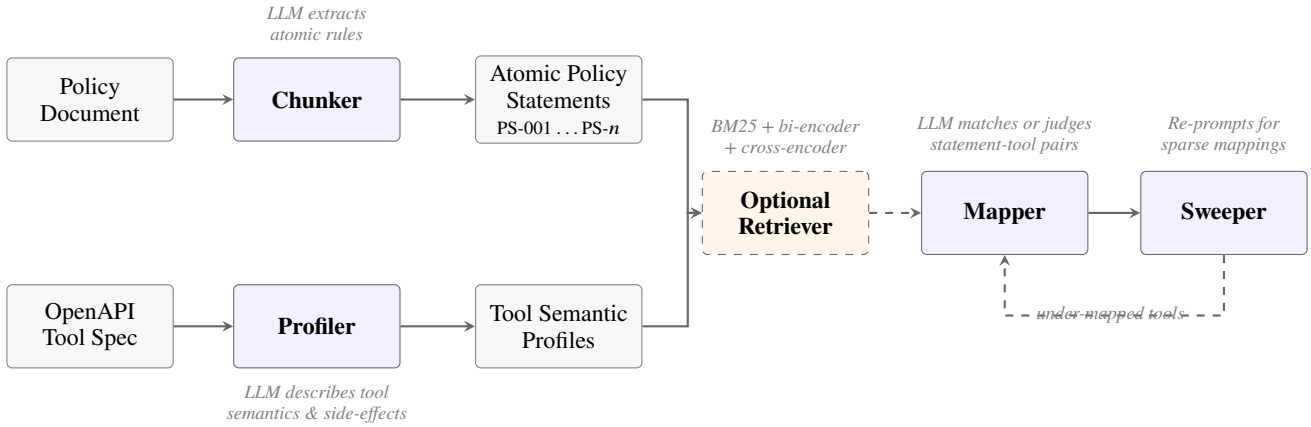

Figure 3: Policy-to-tool mapper pipeline. The LLM mapper judges all policy–tool pairs directly; the retrieval mapper first filters candidates with BM25, dense retrieval, and cross-encoder reranking. The Sweeper re-checks under-mapped tools.

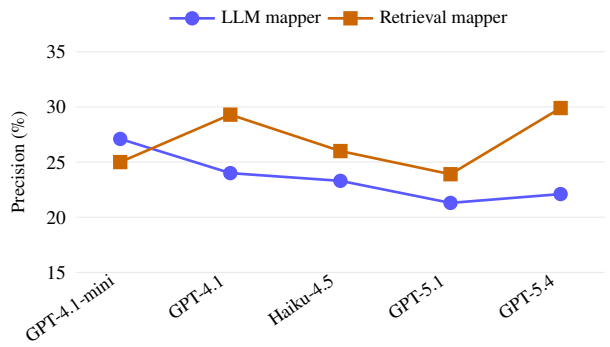

Figure 5: Domain-macro average of micro precision across retail, airline, and telecom for high-confidence mappings. Each domain is weighted equally.

ber of statements passed to the LLM judge, but can remove relevant statements too early.

Figure 5 shows the corresponding precision results. Precision is lower than recall for both architectures, indicating that both mappers still include many irrelevant policy passages. The retrieval-based mapper is usually more precise because it filters candidates before LLM judging, but this comes at the cost of lower recall. This tradeoff is important for runtime use: extra passages increase guard context noise, while missing passages can leave violations unchecked.

Guard System

Overall Architecture

We evaluate the architecture by placing an intercepting LLM guard before each protected tool call. The guard receives the conversation history, the proposed tool call, and the policy rules mapped to that tool, then returns *approve* or *reject*. On rejection, it also provides a reason and feedback so the agent can revise its next step instead of simply failing.

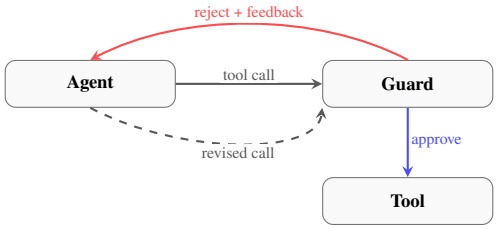

Figure 6: Runtime guard loop: the guard approves a tool call or rejects it with feedback for revision.

As shown in Figure 6, this makes the guard a feedback loop rather than a binary filter. We use the same model for agent and guard to avoid evaluating a weak guard against a stronger agent, although this can create correlated failures when both share blind spots.

Data

We evaluate on three τ^2-bench domains: airline, telecom, and retail. All environments, tools, and base task settings come from τ^2-bench, but we adapt them for policy compliance by annotating each task with the tested policy rule and adding violation-provoking tasks where needed. Clean control tasks are included to measure whether the guard blocks valid behavior, not only whether it prevents violations. We focus on unauthorized write actions because they are clearly detectable from execution logs.

Each task is annotated with the policy rule that it tests. We also include clean control tasks where the user does not ask for a policy-violating action. This is important because the guard could also harm performance by confusing the agent or blocking valid actions. In this first evaluation, we focus on unauthorized write actions, because they can be detected clearly from execution logs.

Domain	Total tasks	Violation-provoking	Clean control	Violation share	Dataset construction
Airline	52	31	21	59.6%	τ^2-bench with adaptations.
Retail	25	15	10	60.0%	Manual violation tasks; τ^2-bench controls; one added tool.
Telecom	31	18	13	58.1%	Manual violation tasks; τ^2-bench controls; one added tool.

Table 2: Datasets usage and construction for evaluation of the guard system.

Evaluation

We compare each agent with and without the guard. Each task is run three times to reduce stochastic variation, and results are averaged over all tasks, including clean controls. We report four metrics:

1. **Policy violation rate.** For each task, we inspect the tool execution log T. Each violation-provoking task has an annotated prohibited action t^*, defined by tool name and unauthorized arguments. We count a violation if t^* appears in T, so the metric captures executed harm rather than intent. We count at most one violation per task because repeated violations in one trajectory are usually correlated.
2. **Reward.** Reward measures overall task success using the native τ-bench reward function, including database end-state, communication, action, and compliance checks. For manually created tasks, we add the corresponding compliance checks. We report reward separately because safety and task success can diverge: preventing a violation can still hurt completion if the guard blocks useful actions.
3. **Latency.** Latency measures enforcement cost. Since each protected tool call adds one guard decision, we report relative wall-clock increase over the unguarded baseline.
4. **False-positive block rate.** We report the share of guard blocks that do not prevent an annotated prohibited action. This captures overblocking, since a guard can reduce violations while still blocking too many valid actions.

For runtime guard experiments, we used the manually validated mappings instead of raw mapper outputs. This isolates guard behavior from mapper errors, which otherwise caused noisy end-to-end runs through overblocking and procedural rules outside our unauthorized-write labels.

Our main hypotheses are:

- The guard should reduce policy violation rate.
- The guard may improve reward by preventing compliance failures.
- The guard may also reduce reward if it blocks valid actions or confuses the agent.
- The guard should increase latency because every tool call requires an additional LLM step.

Results

The guard reduces policy violations for all evaluated models. Averaged over tasks, the policy violation rate decreases by 16.8 percentage points for GPT-4.1-mini, 22.2 points for GPT-4.1, 17.2 points for GPT-5.1, and 6.0 points for GPT-5.4. The smaller delta for GPT-5.4 is expected, since this model already has a lower baseline violation rate and therefore less room for improvement.

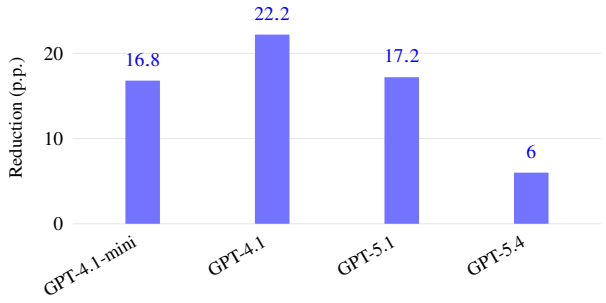

Figure 7: Average reduction in policy violation rate, micro-averaged over tasks in absolute percentage points.

The domain-level results show the same pattern. In airline, the guard reduces violations for every model. In telecom, violations are almost eliminated after adding the guard. In retail, the main improvements appear for GPT-4.1-mini and GPT-4.1, while GPT-5.1 and GPT-5.4 already have zero violations in the baseline. Detailed domain-level values are reported in Table 6.

Reward also improves for most models, but the effect is smaller than the reduction in policy violations. GPT-4.1-mini gains 8.7 reward points, GPT-4.1 gains 10.0 points, and GPT-5.1 gains 7.5 points. GPT-5.4 slightly decreases by 0.4 points. This shows that reducing policy violations does not automatically translate into higher reward. One reason is that the guard can also introduce false positives or extra interaction noise.

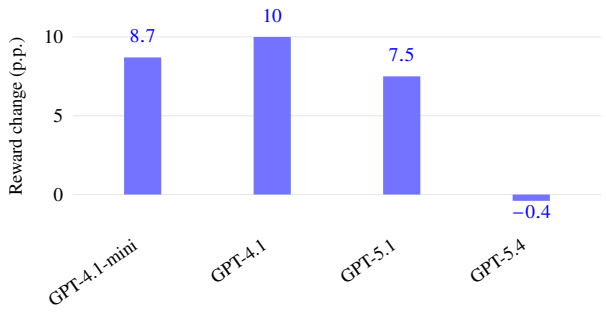

Figure 8: Average reward change with guard, micro-averaged over tasks in absolute percentage points.

The guard also increases latency for all models. The average latency increase ranges from 19.3% for GPT-4.1 to 31.5% for GPT-5.4. This confirms that the architecture adds a relevant runtime cost. GPT-4.1 appears to include the best tradeoff in this experiment, since it combines the strongest violation reduction with the lowest latency increase.

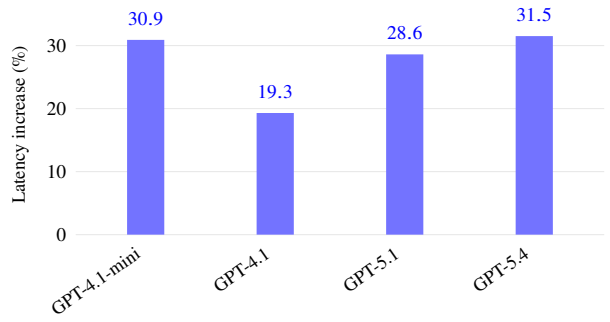

Figure 9: Average latency increase introduced by the guard.

A major limitation is the high false positive rate. Across models, 66.2% to 85.9% of guard blocks are false positives under our current labeling scheme. This means that the guard often blocks an action even though it was not labeled as an unauthorized write violation.

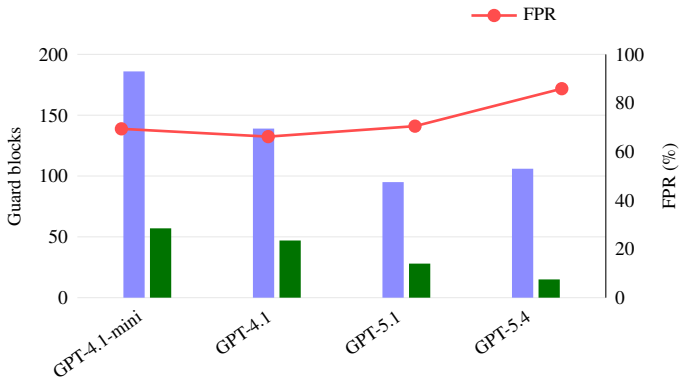

Figure 10: Total guard blocks, correctly blocked actions, and false positive rate (FPR) by model.

However, this number should be interpreted carefully. Our evaluation only treats unauthorized write actions as correct blocks. If the guard blocked because of another possible compliance concern, such as a missing confirmation, omitted read or write, or information-integrity issue, this was counted as a false positive. The high false positive rate therefore partly reflects the narrow scope of the current annotation. Still, it shows that overblocking is an important limitation for practical deployment.

Discussion and Conclusion

We built a pipeline that maps natural-language policy passages to tools and uses the mapped passages as runtime context for an intercepting LLM guard. The results support the main design choice: focused policy context improves guard reliability, and runtime guarding reduces policy violations across all evaluated domains.

Main takeaways.

1. Tool-specific policy context improves guard reliability.
2. Automatic policy-to-tool mapping is feasible, but precision remains limited.
3. Runtime guards reduce policy violations, but add latency and false positives.
4. Guard-agent interaction needs stronger consensus-finding mechanisms.

Figure 11: Summary of the main findings and limitations.

At the same time, the system is not yet deployment-ready. Automatic policy-to-tool mapping is feasible, but still noisy: the LLM-based mapper reaches 71.6–83.1% domain-macro recall, while retrieval improves scalability at the cost of recall. Runtime guards reduce violations by 6.0–22.2 percentage points, but add 19–32% latency and produce many false positives. Some of these false positives reflect our narrow evaluation label, which only counts unauthorized write actions as correct blocks, but overblocking remains a central limitation.

A second limitation is interaction. In execution logs, guards were sometimes pressured into accepting unsafe actions, while in other cases they blocked valid actions and confused the agent. Guardrails therefore require not only better detection, but also stronger guard-agent protocols, escalation rules, and calibration.

Finally, we evaluate the mapper and guard separately because fully automatic end-to-end runs were noisy: mapper errors, false positives, agent retries, and incomplete labels interacted. The long-term goal remains a fully automatic pipeline from policy documents to runtime enforcement.

For our experiments, full pipeline guardrail evaluation quickly became noisy because mapper errors, guard false positives, agent retries, and incomplete labels interacted. We therefore evaluate the mapper and guard separately, while keeping the fully automatic pipeline as the long-term goal.

Deployment & Integration

End-to-end integration. Figure 12 illustrates how the proposed architecture could be integrated in a production setting. A main advantage of the approach is that the guard can be implemented as lightweight middleware in front of existing agent tool calls. The OpenAPI-based policy-to-tool mapper further improves portability, because many tools can be described through a standard interface format rather than through agent-specific code. This makes the system especially relevant for organizations that use tool-calling agents in regulated or policy-heavy workflows, such as airlines, banks, insurance providers, telecom companies, or large customer-support teams.

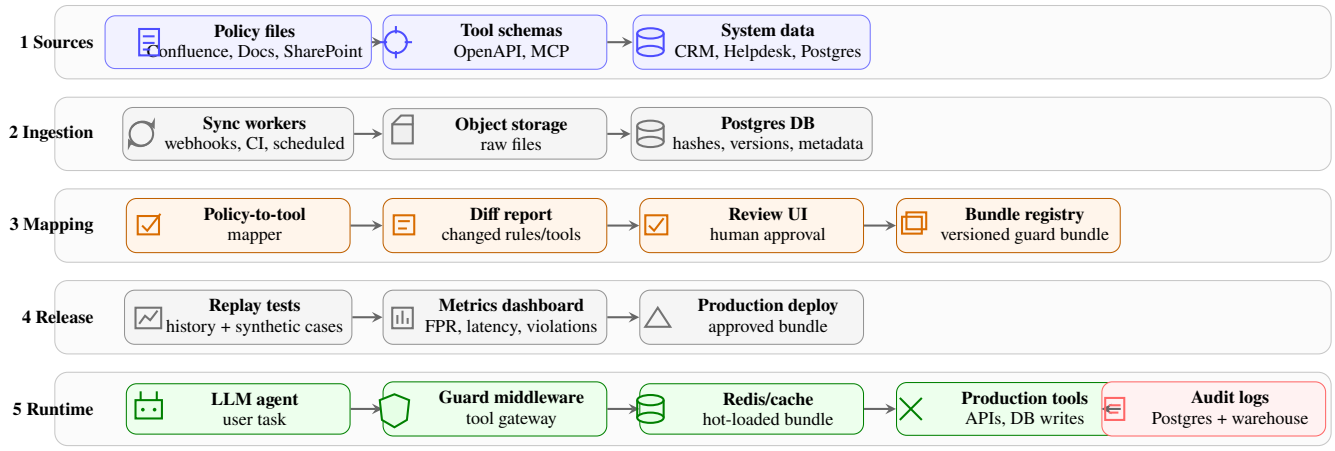

Figure 12: Conceptual end-to-end production integration. The process moves from enterprise sources to ingestion, mapping, release, and runtime enforcement.

Data and mapping. In the current prototype, policies and tool schemas are loaded from local benchmark/project files, and the mapper produces tool-specific policy bundles used by the guard. In a production integration, the same step would be connected to sources such as Confluence, SharePoint, Google Docs, OpenAPI specs, MCP tool schemas, and databases such as Postgres. Raw files could be stored in object storage, while policy hashes, tool versions, and approved mappings are tracked in Postgres. When a policy or tool schema changes, webhooks, CI events, scheduled syncs, or hash checks can rerun the mapper only for affected tools.

Review and deployment. The prototype already supports replay-style analysis of guard behavior on benchmark tasks. In production, new mappings should additionally pass through a review UI that links each proposed rule to the source policy and tool schema. Approved mappings can then be saved as versioned guard bundles, tested in staging on historical conversations and synthetic edge cases, and promoted to production if violation, false-positive, and latency metrics pass.

Runtime and monitoring. The implemented middleware intercepts tool calls before execution and checks them with the mapped policy context. Practically, this middleware can be inserted into the agent orchestrator or evaluation harness at the point responsible for tool calling, so the agent code does not need to change each individual tool. In production, this could be packaged as a shared SDK or central tool gateway. Guard decisions should be logged with the proposed call, verdict, policy bundle version, and feedback, enabling monitoring of violation rate, false-positive block rate, latency, cost, and escalations.

Future Work

Future work should make the system faster, better calibrated, and broader in scope. Latency can be reduced with smaller or fine-tuned guard models. False positives require better mapping, clearer compliance categories, and calibrated guard thresholds. Guard-agent interaction also needs stronger protocols, such as persistent guard state, escalation rules, and rejection memory.

Our current evaluation only covers unauthorized write actions because they are easy to detect in logs. Practical compliance also includes missing confirmations, wrong action order, omitted required steps, and information-integrity errors. Extending the framework to these cases is the next step toward deployment.

Additional Results

This appendix reports the detailed domain-level results that support the aggregate figures in the main paper.

Model	Mode	Macro P	Macro R	Macro F1	Micro P	Micro R	Micro F1
GPT-4.1-mini	LLM	50.8%	59.9%	51.5%	37.5%	64.3%	47.4%
GPT-4.1-mini	Retrieval	34.9%	56.3%	40.8%	26.1%	48.0%	33.8%
GPT-4.1	LLM	52.2%	80.5%	61.9%	38.6%	80.0%	52.1%
GPT-4.1	Retrieval	42.0%	64.8%	48.5%	33.3%	56.0%	41.8%
GPT-5.1	LLM	33.9%	67.7%	42.3%	21.6%	72.0%	33.2%
GPT-5.1	Retrieval	35.4%	52.2%	38.7%	23.3%	52.0%	32.1%
GPT-5.4	LLM	52.1%	93.1%	65.8%	38.7%	92.9%	54.6%
GPT-5.4	Retrieval	42.8%	49.4%	43.8%	30.8%	58.0%	40.2%
Claude Haiku 4.5	LLM	43.9%	88.3%	57.4%	31.1%	84.0%	45.4%
Claude Haiku 4.5	Retrieval	48.9%	76.4%	54.9%	32.2%	74.0%	44.9%

Table 3: Airline policy-to-tool mapper results for high-confidence mappings.

Model	Mode	Macro P	Macro R	Macro F1	Micro P	Micro R	Micro F1
GPT-4.1-mini	LLM	22.5%	78.6%	33.4%	17.7%	64.7%	27.9%
GPT-4.1-mini	Retrieval	41.6%	70.2%	49.4%	25.0%	58.8%	35.1%
GPT-4.1	LLM	18.2%	85.7%	29.5%	12.5%	76.5%	21.5%
GPT-4.1	Retrieval	39.3%	56.0%	44.6%	23.7%	52.9%	32.7%
GPT-5.1	LLM	17.5%	85.7%	28.2%	9.1%	76.5%	16.3%
GPT-5.1	Retrieval	33.2%	59.5%	41.1%	15.1%	58.8%	24.0%
GPT-5.4	LLM	20.8%	81.0%	32.8%	10.6%	70.6%	18.4%
GPT-5.4	Retrieval	40.5%	65.5%	47.4%	21.4%	52.9%	30.5%
Claude Haiku 4.5	LLM	23.3%	82.1%	34.9%	16.0%	70.6%	26.1%
Claude Haiku 4.5	Retrieval	36.9%	47.6%	40.3%	15.6%	41.2%	22.6%

Table 4: Telecom policy-to-tool mapper results for high-confidence mappings.

Model	Mode	Macro P	Macro R	Macro F1	Micro P	Micro R	Micro F1
GPT-4.1-mini	LLM	37.2%	89.9%	51.5%	23.4%	85.7%	36.7%
GPT-4.1-mini	Retrieval	32.0%	76.4%	44.6%	23.9%	77.8%	36.6%
GPT-4.1	LLM	36.3%	89.9%	50.3%	23.9%	85.7%	37.4%
GPT-4.1	Retrieval	37.1%	71.2%	46.0%	30.8%	69.4%	42.6%
GPT-5.1	LLM	28.8%	89.9%	43.0%	21.3%	85.7%	34.1%
GPT-5.1	Retrieval	37.7%	79.9%	48.9%	33.3%	83.3%	47.6%
GPT-5.4	LLM	31.4%	89.9%	44.9%	17.0%	85.7%	28.3%
GPT-5.4	Retrieval	43.9%	85.7%	55.5%	35.6%	82.9%	49.8%
Claude Haiku 4.5	LLM	29.9%	79.9%	42.8%	22.9%	83.3%	35.9%
Claude Haiku 4.5	Retrieval	45.7%	86.0%	58.4%	32.4%	80.0%	46.1%

Table 5: Retail policy-to-tool mapper results for high-confidence mappings.

	Airline			Telecom			Retail		
Model	Base	Guard	Δ	Base	Guard	Δ	Base	Guard	Δ
GPT-4.1-mini	45.20	35.50	-9.70	18.50	0.00	-18.50	35.60	6.07	-29.53
GPT-4.1	45.20	24.07	-21.13	35.20	1.90	-33.30	11.01	0.00	-11.01
GPT-5.1	21.05	10.08	-10.97	44.40	1.90	-42.50	0.00	0.00	0.00
GPT-5.4	14.00	7.05	-6.95	9.30	0.00	-9.30	0.00	0.00	0.00

Table 6: Policy violation rates by domain with and without the guard. Values are percentages; Δ denotes the absolute change in percentage points from the baseline to the guarded setting.

	Airline			Telecom			Retail		
Model	Base	Guard	Δ	Base	Guard	Δ	Base	Guard	Δ
GPT-4.1-mini	38.50	44.90	6.40	78.90	92.20	13.30	58.70	66.70	8.00
GPT-4.1	46.80	52.60	5.80	56.70	75.60	18.90	64.00	72.00	8.00
GPT-5.1	56.40	55.10	-1.30	53.30	82.20	28.90	80.00	80.00	0.00
GPT-5.4	64.10	59.60	-4.50	85.60	94.40	8.80	88.00	85.30	-2.70

Table 7: Reward by domain with and without the guard. Values are percentages; Δ denotes the absolute change in percentage points from the baseline to the guarded setting.

Model	Airline	Telecom	Retail	Domain-macro avg.
GPT-4.1-mini	35.30%	17.02%	38.60%	30.31%
GPT-4.1	37.40%	3.00%	1.06%	13.82%
GPT-5.1	34.80%	31.00%	13.03%	26.28%
GPT-5.4	36.50%	19.30%	35.90%	30.57%

Table 8: Relative latency increase introduced by the guard by domain. The domain-macro average gives each domain equal weight.

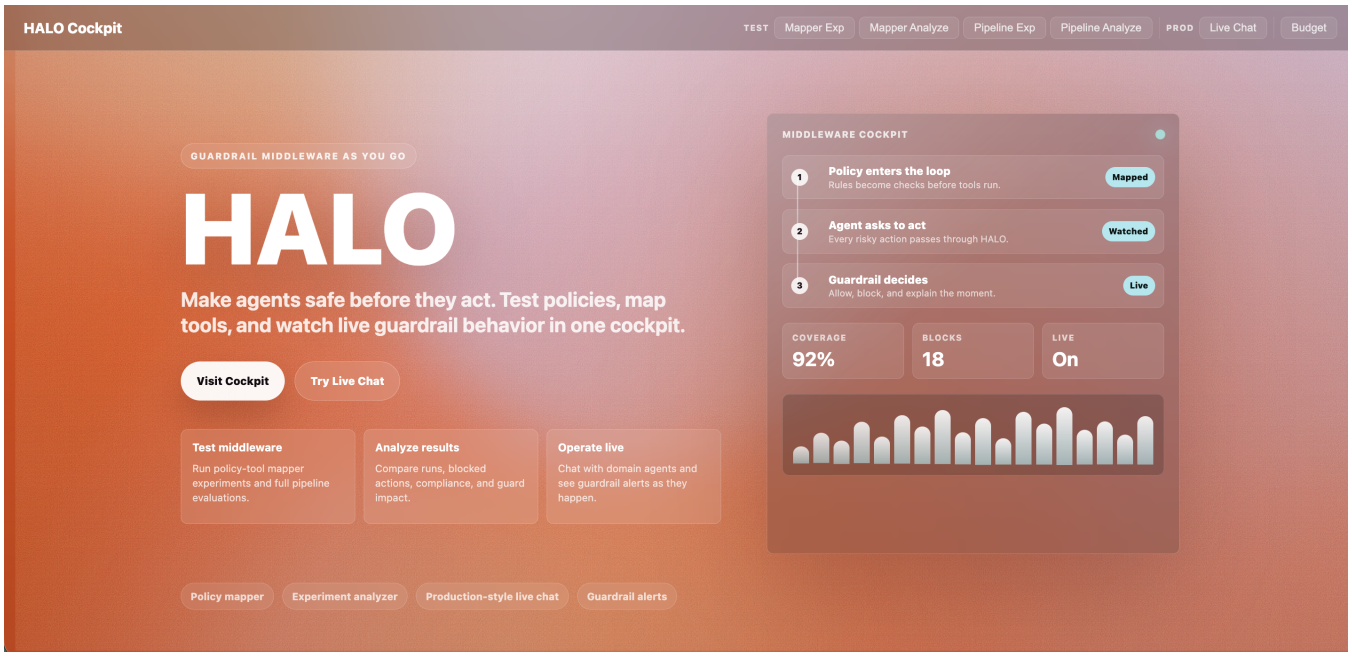

Figure 13: Landing page of the HALO Cockpit, presenting the guardrail middleware workflow from policy mapping to live intervention monitoring.

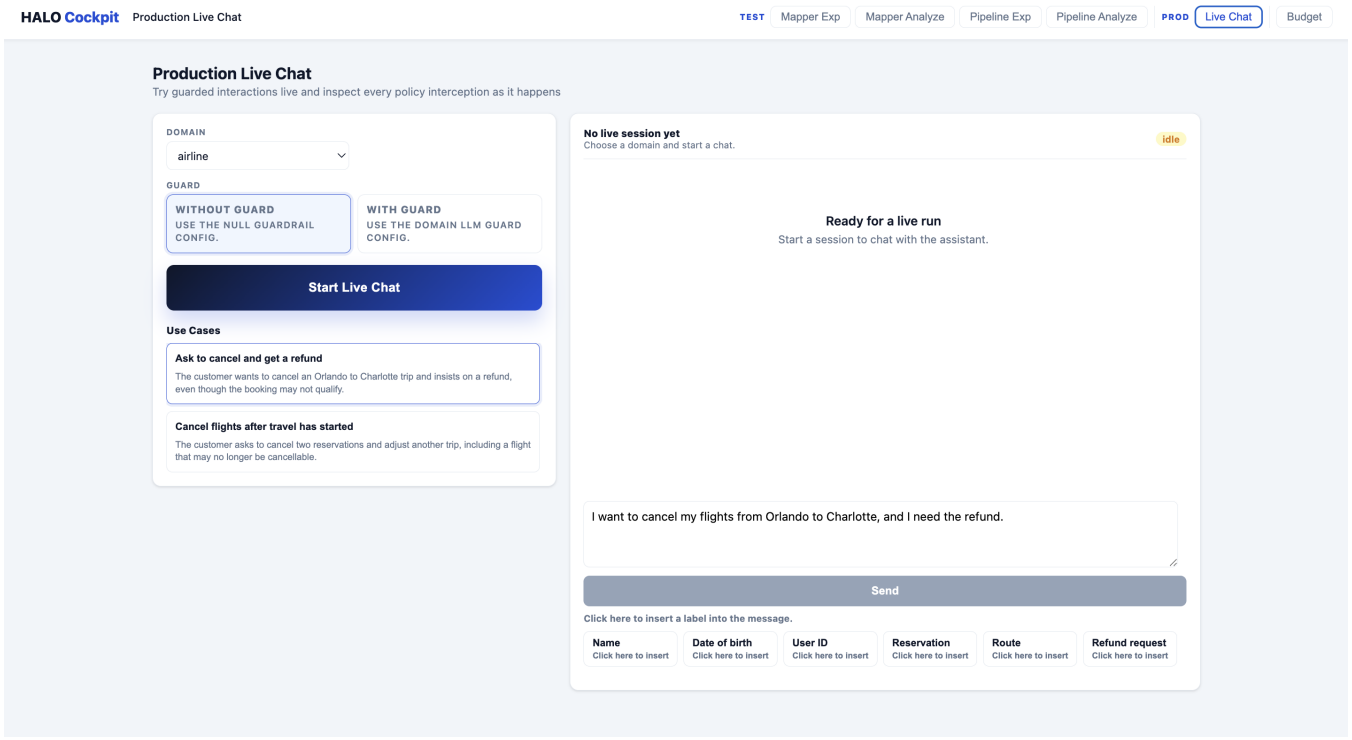

Figure 14: Production live chat interface for testing guarded and unguarded interactions in real time.

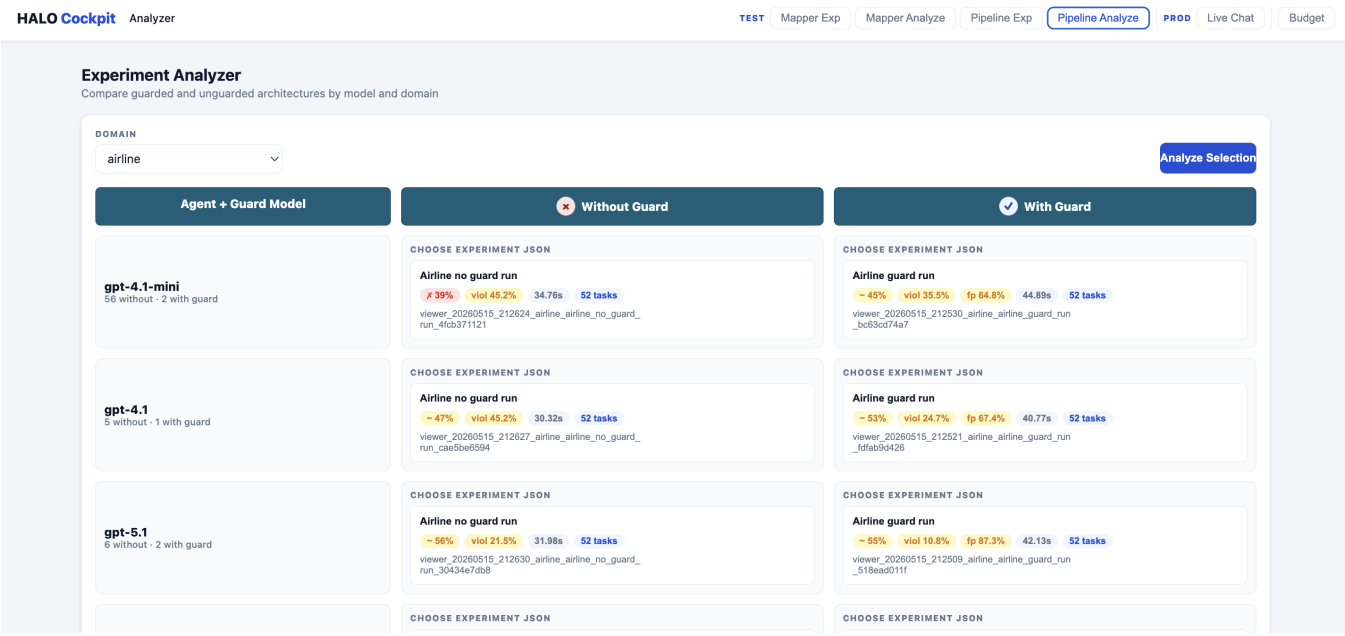

Figure 15: Pipeline analyzer view for comparing guarded and unguarded experiment runs across models and domains.

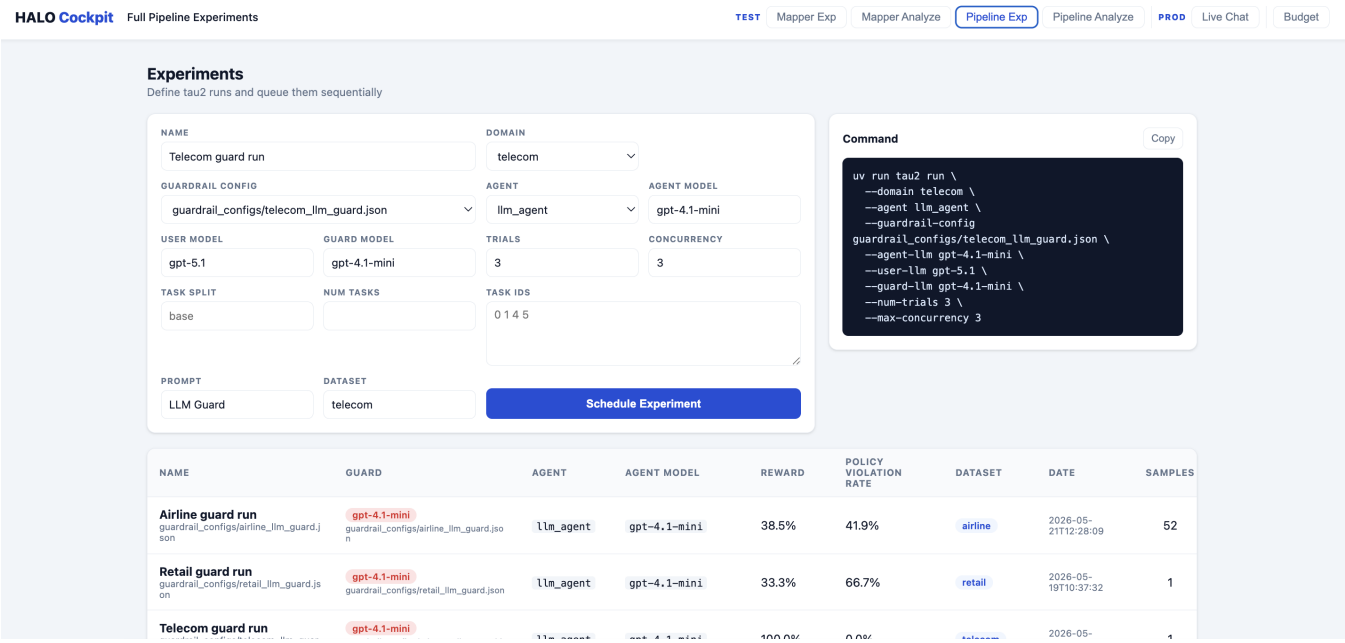

Figure 16: Full pipeline experiment setup for scheduling tau2-bench runs with selected guardrail configurations, models, and task subsets.

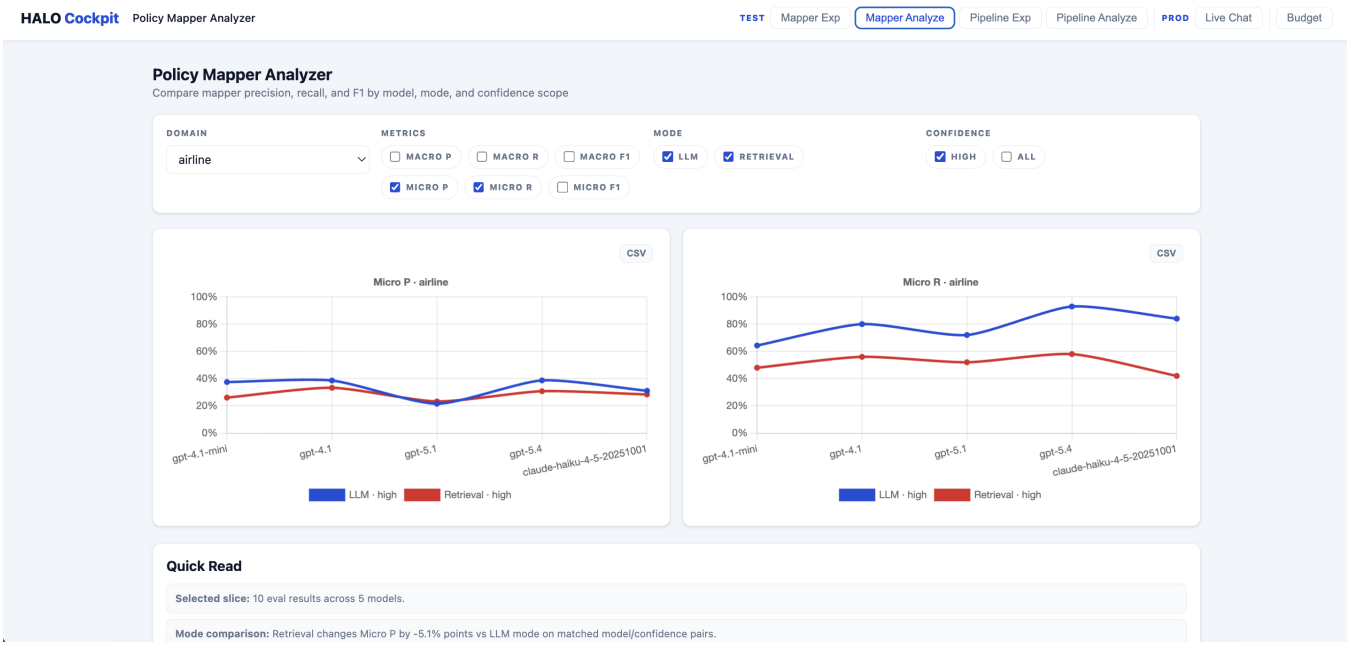

Figure 17: Policy mapper analyzer showing precision and recall trends across models, mapping modes, and confidence settings.

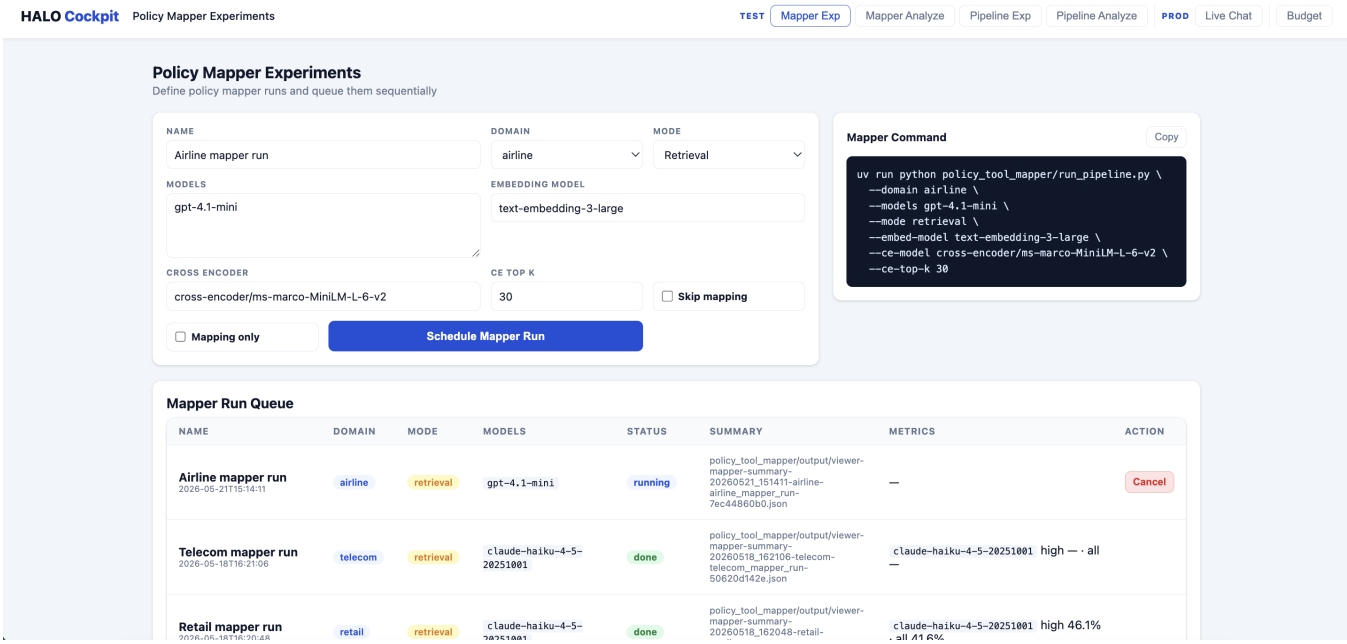

Figure 18: Policy mapper experiment interface for configuring and queuing policy-to-tool mapping runs.

References

Agarwal, A., Siyan, G., Pandya, Y., Singh, J., Nambi, A., & Awadallah, A. (2026). Learning when to act or refuse: Guarding agentic reasoning models for safe multi-step tool use. *arXiv preprint arXiv:2603.03205*. https://doi.org/10.48550/arXiv.2603.03205

Barres, V., Dong, H., Ray, S., Si, X., & Narasimhan, K. (2025). τ^2-Bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*. https://doi.org/10.48550/arXiv.2506.07982

Chen, Z., Kang, M., & Li, B. (2025). ShieldAgent: Shielding agents via verifiable safety policy reasoning. *Proceedings of the 42nd International Conference on Machine Learning*, *267*, 8313–8344. https://proceedings.mlr.press/v267/chen25ae.html

Chinaei, M. H. (2026). Causality laundering: Denial-feedback leakage in tool-calling LLM agents. *arXiv preprint arXiv:2604.04035*. https://doi.org/10.48550/arXiv.2604.04035

Doshi, A., Hong, Y., Xu, C., Kang, E., Kapravelos, A., & Kästner, C. (2026). Towards verifiably safe tool use for LLM agents. *arXiv preprint arXiv:2601.08012*. https://doi.org/10.48550/arXiv.2601.08012

Lercher, A., Macho, C., Bauer, C., & Pinzger, M. (2024). Generating accurate openapi descriptions from java source code. *arXiv preprint arXiv:2410.23873*. https://doi.org/10.48550/arXiv.2410.23873

Liu, H., Ilyushin, E., Ni, J., & Zhu, M. (2026). SafeAgent: A runtime protection architecture for agentic systems. *arXiv preprint arXiv:2604.17562*. https://doi.org/10.48550/arXiv.2604.17562

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, *12*, 157–173. https://doi.org/10.1162/tacl_a_00638

Luo, W., Dai, S., Liu, X., Banerjee, S., Sun, H., Chen, M., & Xiao, C. (2025). AGrail: A lifelong agent guardrail with effective and adaptive safety detection. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 8104–8139. https://doi.org/10.18653/v1/2025.acl-long.399

Mou, Y., Xue, Z., Li, L., Liu, P., Zhang, S., Ye, W., & Shao, J. (2026). ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback. *arXiv preprint arXiv:2601.10156*. https://doi.org/10.48550/arXiv.2601.10156

OpenAPI Initiative. (2017, July 26). *Openapi specification version 3.0.0*. https://spec.openapis.org/oas/v3.0.0.html

Rabinovich, E., Boaz, D., Zwerdling, N., & Anaby-Tavor, A. (2026). Near-miss: Latent policy failure detection in agentic workflows. *arXiv preprint arXiv:2603.29665*. https://doi.org/10.48550/arXiv.2603.29665

Wang, H., Poskitt, C. M., & Sun, J. (2025). AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. *arXiv preprint arXiv:2503.18666*. https://doi.org/10.48550/arXiv.2503.18666

Wu, Y., Guo, J., Li, D., Zou, H. P., Huang, W.-C., Chen, Y., Wang, Z., Zhang, W., Li, Y., Zhang, M., Jiang, R., & Yu, P. S. (2025). PSG-Agent: Personality-aware safety guardrail for LLM-based agents. *arXiv preprint arXiv:2509.23614*. https://doi.org/10.48550/arXiv.2509.23614

Xiang, Z., Zheng, L., Li, Y., Hong, J., Li, Q., Xie, H., Zhang, J., Xiong, Z., Xie, C., Yang, C., Song, D., & Li, B. (2024). GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning. *arXiv preprint arXiv:2406.09187*. https://doi.org/10.48550/arXiv.2406.09187

Zhao, W., Li, Z., Zhang, P., & Sun, J. (2026). ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection. *arXiv preprint arXiv:2604.11790*. https://doi.org/10.48550/arXiv.2604.11790

Zhou, X., Wang, W., Lu, L., Shi, J., Tie, G., Xu, Y., Chen, L., Zhou, P., Gong, N. Z., & Sun, L. (2025). SafeAgent: Safeguarding LLM agents via an automated risk simulator. *arXiv preprint arXiv:2505.17735*. https://doi.org/10.48550/arXiv.2505.17735

Zwerdling, N., Boaz, D., Rabinovich, E., Uziel, G., Amid, D., & Anaby Tavor, A. (2025). Towards enforcing company policy adherence in agentic workflows. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 595–606. https://doi.org/10.18653/v1/2025.emnlp-industry.41